Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Tasks Performed:

- Deployed the Library contract; Account 1 acted as the librarian.
- Added a book ("Blockchain_Fundamentals" by Vitalik) and checked that `bookCount` returned 1.
- Called `addBook` from a non-librarian account and saw it revert, which tested the `require` check.
- Borrowed the book with `borrowBook(1)` and confirmed `isIssued = true` with the borrower's address.
- Returned it with `returnBook(1)` and confirmed `isIssued = false`.

Theory:

Relevance of require statements – The require statement is a guard condition inside a function. It ensures that only valid inputs or authorized users can run certain code. If the condition fails, execution stops immediately and every state change made in that transaction is reverted, so invalid transactions cannot corrupt blockchain data. In the Library contract it is used for:

- **Access control –** only the librarian may call addBook, e.g. require(msg.sender == librarian, "Only the librarian can add books.").
- **Availability –** a book must be available before it can be borrowed.
- **Verification –** only the user who borrowed a book may return it.

Hence require enforces security, correctness and reliability, and its error messages make debugging easier.

mapping – A key–value structure similar to a hash table, declared as mapping(keyType => valueType). The contract uses mapping(uint => Book) public books; so each Book ID maps to a Book struct (title, author, availability). Mappings have no length and cannot be iterated, which makes direct lookups gas-efficient.

storage – The permanent memory of the contract, kept on the blockchain. Contract-level variables such as librarian, bookCount and the books mapping live in storage. Writing to it costs gas, and updates such as a book's issued status are visible to all users.

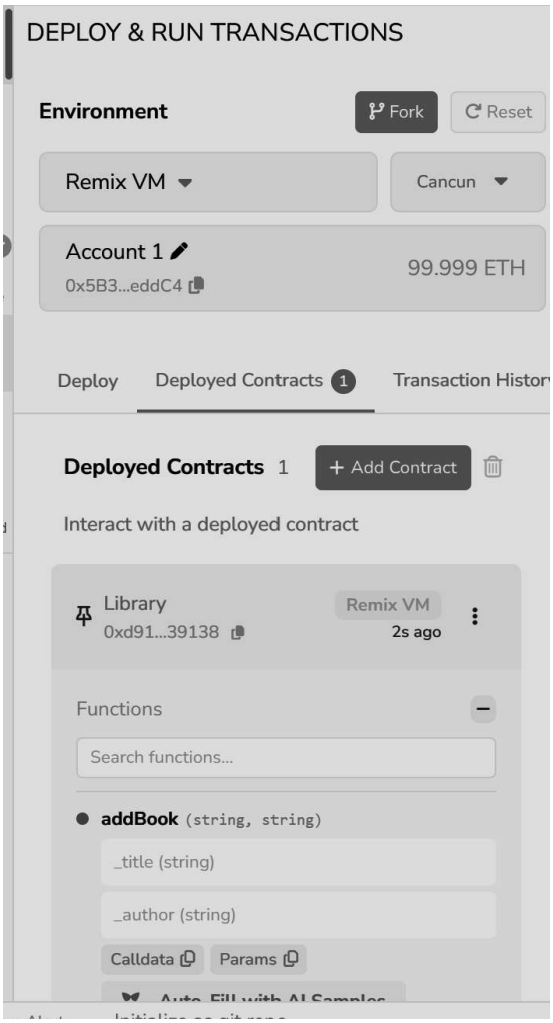
memory – Temporary storage that exists only during a function call and is discarded afterwards. It is cheaper than storage; string memory is used for the title and author passed into addBook before they are saved to the mapping.

string for text flexibility – Fixed-size types such as bytes32 are cheaper but limited to 32 bytes. Book titles and author names vary in length, so the dynamically sized string type is used for readability and flexibility, at a slightly higher gas cost.

Code & Output:

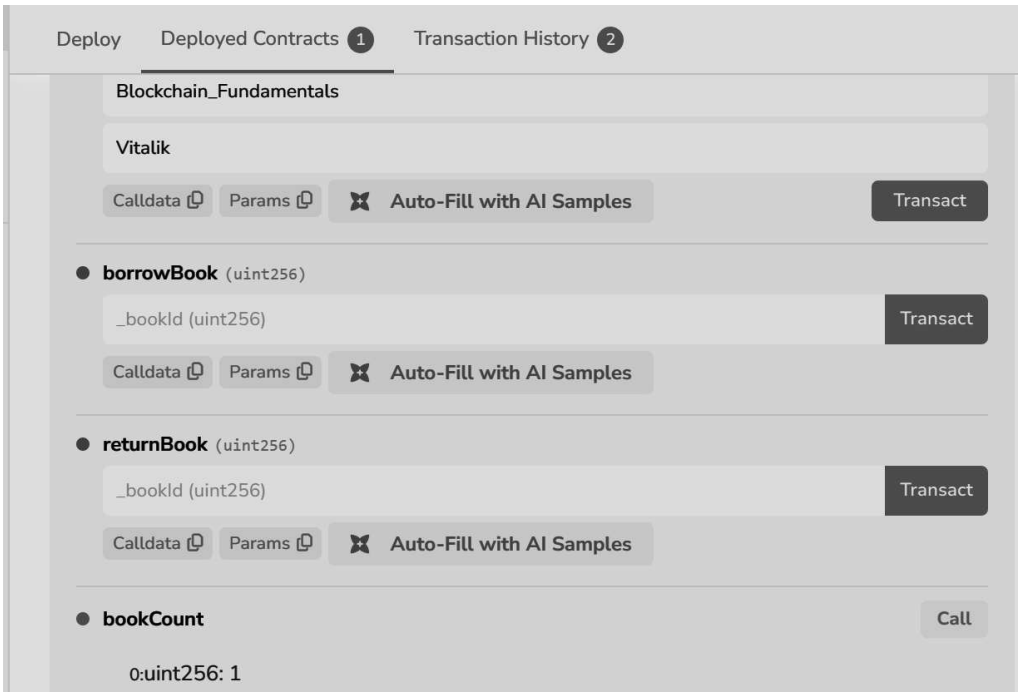
Step 1 — Deploy the Library contract

The Library contract is deployed on the Remix VM (Cancun) from Account 1, which acts as the librarian. It appears under Deployed Contracts with its functions.



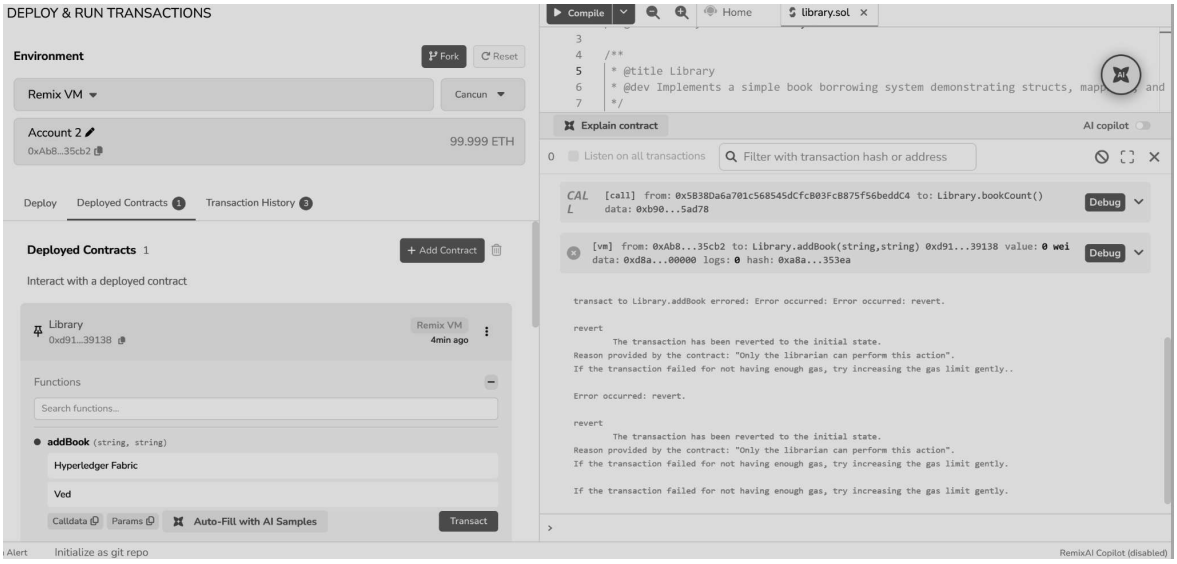
Step 2 — Add a book as the librarian and check bookCount

addBook is called “Blockchain_Fundamentals” and author “Vitalik”. The contract also exposes borrowBook, returnBook and bookCount, and bookCount returns 1.



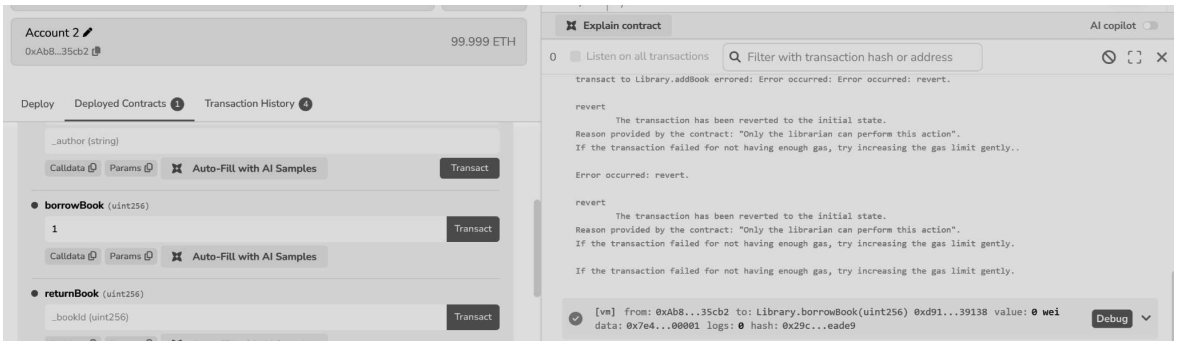
Step 3 — Test access control with require

Account 2 (not the librarian) tries to call addBook with “Hyperledger Fabric” and “Ved”. The transaction reverts with the reason “Only the librarian can perform this action”.



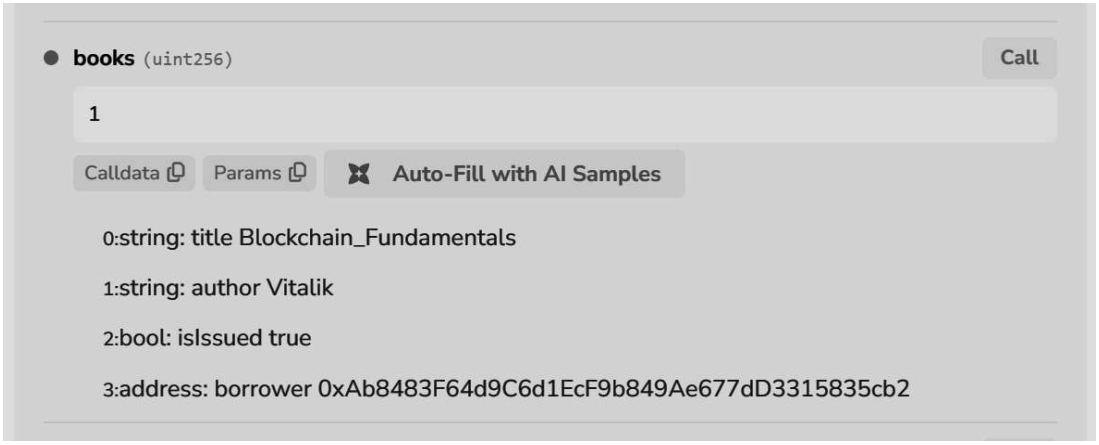
Step 4 — Borrow a book

Account 2 calls borrowBook with _bookId = 1. The transaction succeeds and is logged in the Remix console.



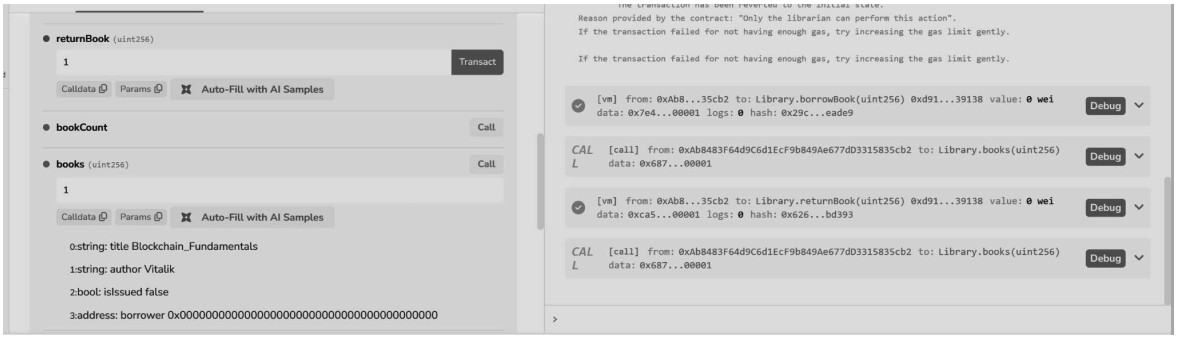
Step 5 — Verify the book status after borrowing

Calling books(1) shows the title, the author, isIssued = true and the borrower's address.



Step 6 — Return the book and verify

Account 2 calls returnBook with _bookId = 1. Calling books(1) again shows isIssued = false and the borrower reset to the zero address.



Conclusion:

The **Decentralized Library** smart contract was successfully compiled, deployed and tested using Solidity and Remix IDE. The experiment demonstrated **role-based access control** (only the librarian can add books) and **user-driven state changes** (borrowing and returning books). It also gave hands-on understanding of **required statements, mapping, storage versus memory, and the dynamically sized string type**.